

Projets NSI Première : travailler en équipe avec Git et GitHub

Sep 30, 2026 · @Jim Pioche

Ce guide accompagne votre groupe de trois élèves, pas à pas, de la création de l'équipe jusqu'à la consigne du projet n°1 de NSI Première. Suivez les sections dans l'ordre : chacune prépare la suivante.

Table des matières

1. Création des groupes de projet
2. Création d'un compte GitHub
3. Création du dépôt sur GitHub
4. Invitation de M. Pioche sur le dépôt
5. Installation de GitHub Desktop sous Windows
6. Parcours Git pas à pas
 - Vocabulaire et schéma d'ensemble
 - Étape 1 : faire son premier commit
 - Étape 2 : modifier un fichier et lire l'historique
 - Étape 3 : annuler une erreur
 - Étape 4 : travailler à plusieurs avec push et pull
 - Étape 5 : les branches
 - Étape 6 : provoquer et résoudre un conflit
 - Étape 7 : règles de travail pendant le projet
 - Aide-mémoire
7. Consigne du projet n°1
 - 7.1 Sujet du projet n°1 : simulation du comportement d'un DAB
 - 7.2 Attendus concernant la programmation
 - 7.3 Modalités de notation

1. Création des groupes de projet

Chaque projet est réalisé par un groupe de 3 élèves, et ces trois élèves partagent un seul dépôt GitHub.

1. Formez votre groupe de 3 élèves.

2. Choisissez le nom de votre projet, sans espace ni accent (exemple : `convertisseurBases`). Il servira à nommer le dépôt à la section 3.
3. Désignez le ou la responsable du dépôt : cette personne crée le dépôt sur GitHub, puis invite les deux autres membres.
4. Complétez la fiche ci-dessous. Les identifiants GitHub se renseignent après la section 2.

Information	À remplir
Nom du projet	
Responsable du dépôt (nom, prénom, identifiant GitHub)	
Élève 2 (nom, prénom, identifiant GitHub)	
Élève 3 (nom, prénom, identifiant GitHub)	

2. Création d'un compte GitHub

Chaque élève crée son propre compte GitHub : c'est ce compte qui signera ses commits et qui permettra d'identifier son travail.

1. Ouvrez github.com et cliquez sur **Sign up**.
2. Saisissez une adresse e-mail que vous consultez régulièrement : GitHub y envoie un code de vérification.
3. Choisissez un mot de passe long, que vous n'utilisez nulle part ailleurs.
4. Choisissez un nom d'utilisateur (*username*) sobre et reconnaissable, par exemple `prenom-nom` . Évitez les pseudos fantaisistes : on doit pouvoir vous identifier.
5. Résolvez la vérification demandée, puis validez le code reçu par e-mail.
6. Notez votre nom d'utilisateur dans la fiche de la section 1 et communiquez-le aux deux autres membres du groupe.

Vérification : en haut à droite de github.com, votre photo de profil s'affiche et un clic dessus montre votre nom d'utilisateur.

3. Création du dépôt sur GitHub

Le ou la responsable du dépôt crée un seul dépôt privé pour le groupe, nommé selon la règle `2026_2027_nsi_prem_projet1__nomDuProjet` .

Exemple pour un projet appelé `convertisseurBases` :

`2026_2027_nsi_prem_projet1__convertisseurBases`. Attention aux deux tirets bas (`__`) avant le nom du projet, et pas d'espace ni d'accent.

1. Connecté à GitHub, cliquez sur le **+** en haut à droite, puis sur **New repository**.
2. **Owner** : laissez votre compte.
3. **Repository name** : saisissez le nom du dépôt, exactement selon la règle ci-dessus.
4. **Description** : une phrase qui décrit votre projet (facultatif).
5. Choisissez **Private** : seules les personnes invitées pourront voir le dépôt.
6. Cochez **Add a README file**.
7. Dans **Add .gitignore**, choisissez le modèle **Python**. Il évite d'envoyer sur GitHub des fichiers temporaires comme `__pycache__`.
8. Laissez la licence vide et cliquez sur **Create repository**.

Vérification : la page du dépôt s'affiche, avec les fichiers `README.md` et `.gitignore`. Les invitations se font à la section suivante.

4. Invitation de M. Pioche sur le dépôt

M. Pioche doit être invité comme collaborateur du dépôt pour pouvoir consulter et évaluer votre travail. La même procédure sert à inviter vos deux coéquipiers.

Pour le ou la responsable du dépôt

1. Ouvrez la page de votre dépôt sur github.com et cliquez sur l'onglet **Settings**.
2. Dans le menu de gauche, cliquez sur **Collaborators**. GitHub peut vous demander de confirmer votre mot de passe.
3. Cliquez sur **Add people**.
4. Saisissez l'identifiant GitHub de M. Pioche (à compléter par M. Pioche : `identifiant-à-compléter`) et sélectionnez-le dans la liste.
5. Cliquez sur le bouton **Add ... to this repository**.
6. Recommencez avec l'identifiant de chacun de vos deux coéquipiers.

Pour chaque personne invitée

Acceptez l'invitation avec le lien du courriel envoyé par GitHub, ou en ouvrant la page du dépôt, où un bandeau propose d'accepter. Une invitation expire au bout de 7 jours.

Vérification : dans **Settings > Collaborators**, les trois autres comptes apparaissent (la mention *Pending invite* disparaît une fois l'invitation acceptée).

5. Installation de GitHub Desktop sous Windows

GitHub Desktop est un logiciel gratuit qui permet d'utiliser Git avec des boutons plutôt qu'en ligne de commande. Chaque élève l'installe sur son poste et récupère une copie du dépôt.

1. Ouvrez desktop.github.com et cliquez sur **Download for Windows**.
2. Lancez le fichier téléchargé (`GitHubDesktopSetup-x64.exe`). L'installation est automatique et, en principe, ne demande pas de droits administrateur.
3. Au premier lancement, cliquez sur **Sign in to GitHub.com**. Le navigateur s'ouvre : connectez-vous, cliquez sur **Authorize desktop**, puis autorisez l'ouverture de l'application.
4. Dans l'écran **Configure Git**, vérifiez vos champs **Name** (prénom et nom) et **Email** (l'adresse de votre compte GitHub), puis cliquez sur **Finish**. Ces informations signent vos commits : elles permettent de savoir qui a écrit quoi. Vous pouvez les modifier dans **File > Options > Git**.
5. Acceptez d'abord l'invitation au dépôt (section 4). Puis cliquez sur **File > Clone repository...**, choisissez l'onglet **GitHub.com**, sélectionnez votre dépôt dans la liste et vérifiez le champ **Local path** : c'est le dossier de votre poste où seront copiés les fichiers. Cliquez sur **Clone**.
6. Vérification : GitHub Desktop affiche votre dépôt. Le bouton **Show in Explorer** ouvre le dossier, qui contient `README.md`.

Sur un poste partagé du lycée, déconnectez-vous en fin de séance avec **File > Options > Accounts > Sign out**. Si le poste peut être réinitialisé, faites un **Push** avant de partir (voir l'étape 4 du parcours Git).

6. Parcours Git pas à pas

Git enregistre l'historique de votre projet et permet de travailler à plusieurs sans s'écraser mutuellement : ce parcours de 7 étapes vous le fait découvrir par la pratique, en faisant de vrais commits sur votre poste.

Chaque étape suit le même schéma : **À faire** dans GitHub Desktop, **Commande Git équivalente**, **À observer**. Les commandes ne sont pas à taper, car GitHub Desktop les exécute pour vous. Si Git est installé sur le poste et que `git --version` répond dans un terminal ouvert dans le dossier du dépôt (menu **Repository > Open in Command Prompt**), vous pouvez aussi les essayer.

Les étapes 1 à 3 se font chacun sur son poste. Les étapes 4 à 6 se font à deux ou à trois, côte à côte. Dans tout le parcours, remplacez `prenom` par votre prénom.

Vocabulaire et schéma d'ensemble

Mot	Signification
Dépôt (<i>repository</i>)	Dossier de projet dont Git conserve tout l'historique
Dépôt local	La copie du dépôt sur votre poste, où vous travaillez
Dépôt distant (<i>origin</i>)	Le dépôt hébergé sur GitHub, partagé par le groupe
Cloner	Copier sur son poste, une première fois, un dépôt GitHub
Commit	Une « photo » de l'état du projet, avec un message, un auteur et une date
Push	Envoyer vos commits locaux vers GitHub
Fetch	Regarder ce qui est nouveau sur GitHub, sans modifier vos fichiers
Pull	Récupérer les nouveaux commits de GitHub et les ajouter à vos fichiers
Branche	Une ligne de travail parallèle ; la branche principale s'appelle <code>main</code>
Merge (fusion)	Rassembler les commits d'une branche dans une autre
Conflit	Deux personnes ont modifié le même endroit d'un fichier : Git demande de choisir

Le circuit d'un travail, de votre clavier jusqu'à GitHub :

```
vos fichiers modifiés --commit--> dépôt local --push--> GitHub (origin)
                                dépôt local <--pull-- GitHub (origin)
```

Un commit reste sur votre poste tant que vous n'avez pas fait de **push**. Vos coéquipiers ne le voient pas avant.

Étape 1 : faire son premier commit

Objectif : enregistrer un premier état de votre projet dans l'historique Git, sur votre poste.

À faire

1. Dans GitHub Desktop, cliquez sur **Show in Explorer** pour ouvrir le dossier du dépôt.
2. Dans ce dossier, créez un fichier `essai_prenom.py` contenant une seule ligne :
`print("Bonjour")`.
3. Revenez dans GitHub Desktop, onglet **Changes** : votre fichier apparaît dans la liste, coché.
4. En bas à gauche, dans le champ **Summary**, écrivez un message court : `Ajout de essai_prenom.py`.
5. Cliquez sur **Commit to main**.

Commande Git équivalente

```
git status
git add essai_prenom.py
git commit -m "Ajout de essai_prenom.py"
```

À observer

- L'onglet **Changes** est maintenant vide : Git n'a plus rien à enregistrer.
- L'onglet **History** affiche votre commit, avec votre nom et l'heure.
- Sur github.com, votre fichier n'existe pas encore : le commit est seulement sur votre poste.

Étape 2 : modifier un fichier et lire l'historique

Objectif : voir exactement ce qui change entre deux commits.

À faire

1. Ouvrez `essai_prenom.py` et ajoutez une deuxième ligne : `print("Au revoir")`. Enregistrez le fichier.
2. Dans l'onglet **Changes**, cliquez sur le nom du fichier : la ligne ajoutée est en vert, et une ligne supprimée serait en rouge.
3. Faites un commit avec le message `Ajout du message d'au revoir`.
4. Modifiez encore le fichier (par exemple, changez `Bonjour` en `Salut`) et faites un troisième commit avec un message précis.
5. Ouvrez l'onglet **History** et cliquez tour à tour sur chacun de vos trois commits.

Commande Git équivalente

```
git diff
git log --oneline
```

À observer

- Chaque commit est une photo complète du projet : en cliquant sur un commit de l'historique, vous voyez uniquement ce qu'il a changé.
- Un bon message de commit est court, précis, et dit ce qui a changé : `Ajout de la fonction est_bissextile` est utile, `modif` ne l'est pas.
- Un commit = une idée. Évitez les commits géants qui mélangent plusieurs modifications sans lien.

Étape 3 : annuler une erreur

Objectif : savoir revenir en arrière, avant un commit puis après un commit.

À faire

1. Dans `essai_prenom.py`, supprimez tout le contenu et enregistrez. Dans l'onglet **Changes**, constatez que tout le fichier est en rouge.
2. Faites un clic droit sur le fichier, puis **Discard changes...** et confirmez : le fichier retrouve son contenu du dernier commit.
3. Ajoutez maintenant volontairement une ligne fausse (`print(Bonjour)`), puis faites un commit `Ajout d'une ligne (erreur)`.
4. Ouvrez l'onglet **History**, faites un clic droit sur ce commit, puis **Revert changes in commit**.
5. Observez l'historique.

Commande Git équivalente

```
git restore essai_prenom.py
git revert <identifiant du commit>
```

À observer

- *Discard* concerne les modifications **non commitées**. Elles sont perdues : c'est une raison de plus de commiter souvent.
- *Revert* concerne un commit déjà fait. Il n'efface rien de l'historique : Git ajoute un **nouveau commit** qui annule les changements du précédent.

Étape 4 : travailler à plusieurs avec push et pull

Objectif : échanger vos commits avec vos coéquipiers par l'intermédiaire de GitHub.

Nommons A, B et C les trois membres du groupe. Chacun a déjà des commits locaux sur son fichier `essai_prenom.py`.

À faire

1. **A** clique sur **Push origin** (en haut de la fenêtre). Sur github.com, A ouvre la page du dépôt : le fichier `essai_prenom.py` (le fichier de A) est apparu. L'onglet **Commits** liste ses commits.
2. **B** clique sur **Fetch origin**. Le bouton devient **Pull origin** : B clique dessus. Le fichier de A arrive dans le dossier de B.
3. **B** clique ensuite sur **Push origin** pour envoyer ses propres commits.
4. **C** fait la même chose : **Fetch origin**, **Pull origin**, puis **Push origin**.
5. **A** fait enfin **Fetch origin** puis **Pull origin** pour récupérer les fichiers de B et de C.

Commande Git équivalente

```
git push
git fetch
git pull
```

À observer

- Si B essaie de faire un push avant un pull, GitHub Desktop le refuse : le dépôt distant contient des commits que B n'a pas encore.
- **Fetch** se contente de vérifier ; **Pull** modifie réellement vos fichiers.
- Sur github.com, l'onglet **Commits** affiche l'auteur de chaque commit : c'est là que l'on suit le travail de chacun.

Étape 5 : les branches

Objectif : tester une idée sans toucher à la version principale du projet.

À faire

1. Vérifiez que **Current Branch** (en haut) affiche `main` . Faites un **Pull origin** si besoin.
2. Cliquez sur **Current Branch**, puis **New Branch**. Nommez-la `essai-prenom` et cliquez sur **Create branch**.
3. Dans `essai_prenom.py` , ajoutez une ligne `print("Test sur ma branche")` et faites un commit.
4. Cliquez sur **Current Branch** et choisissez `main` : regardez le fichier dans l'Explorateur, la ligne ajoutée a disparu.
5. Revenez sur `essai-prenom` : la ligne est de retour.
6. Repassez sur `main` , puis cliquez sur le menu **Branch > Merge into current branch...** et choisissez `essai-prenom` . La ligne est maintenant dans `main` .
7. Faites un **Push origin**. Les autres membres font un **Pull origin**.

Commande Git équivalente

```
git switch -c essai-prenom
git switch main
git switch essai-prenom
git merge essai-prenom
```

À observer

- Changer de branche change réellement les fichiers de votre dossier, sans rien perdre.
- Une branche permet d'essayer une idée : si elle ne convient pas, `main` reste intact.

Étape 6 : provoquer et résoudre un conflit

Objectif : ne pas être surpris quand deux personnes modifient le même endroit. Cela arrive dans tous les projets, et Git sait vous aider.

Cette étape se fait à deux (A et B). Le troisième membre observe, puis les rôles tournent et on recommence.

À faire

1. **A** crée un fichier `conflit.py` contenant une seule ligne : `message = "Bonjour"`. A fait un commit puis un **Push origin**. **B** fait un **Pull origin**.
2. **A** change la ligne en `message = "Salut"` et fait un commit, **sans push**.
3. **B**, de son côté, change la même ligne en `message = "Coucou"` et fait un commit, **sans push**.
4. **A** clique sur **Push origin** : l'envoi réussit.
5. **B** clique sur **Push origin** : GitHub Desktop le refuse. B clique alors sur **Pull origin** : un message annonce un **conflit** sur `conflit.py`.
6. **B** clique sur **Open in Visual Studio Code** (ou dans l'éditeur proposé). Le fichier contient des marqueurs :

```
<<<<<<< HEAD
message = "Coucou"
=====
message = "Salut"
>>>>>>> origin/main
```

7. Entre les marqueurs `<<<<<<<` et `>>>>>>>`, la version de B est au-dessus de `=====` et la version de A en dessous. Choisissez (ensemble !) la version à garder, supprimez les trois lignes de marqueurs et enregistrez.
8. Dans GitHub Desktop, cliquez sur **Commit merge**, puis sur **Push origin**.
9. **A** fait un **Pull origin** et constate le résultat.

Commande Git équivalente

```
git pull
(modifier le fichier à la main)
git add conflit.py
git commit
git push
```

À observer

- Git ne décide pas à votre place : il vous montre les deux versions et attend votre choix.
- Pour éviter les conflits : répartissez-vous les fonctions ou les fichiers, faites un **Pull** avant de commencer, et poussez souvent.

Étape 7 : règles de travail pendant le projet

Appliquez ces règles à chaque séance de projet : elles évitent la plupart des problèmes et rendent votre travail lisible par tous.

Règles à respecter

1. **En début de séance**, faites **Fetch origin** puis **Pull origin** pour partir de la dernière version du groupe.
2. **Répartissez-vous le travail avant de commencer**, de façon que deux personnes ne modifient jamais le même fichier : c'est la meilleure façon d'éviter les conflits.
3. **Commitez souvent** : un commit = une idée, avec un message précis.
4. **Vérifiez que vos commits portent votre nom** (configuré à la section 5) : c'est ce qui permet d'évaluer la production de chacun.
5. **En fin de séance**, lancez votre programme une fois pour vérifier qu'il démarre, faites un **Push origin**, puis vérifiez sur github.com (onglet **Commits**) que vos commits sont arrivés.
6. **Si vous utilisez une IA**, documentez-la dans le commit selon la règle de la section 7.3.

Aide-mémoire

Je veux...	Dans GitHub Desktop	Commande Git
Voir ce qui a changé	Onglet Changes	<code>git status</code>
Voir le détail d'une modification	Clic sur le fichier	<code>git diff</code>
Enregistrer un commit	Commit to main	<code>git add .</code> puis <code>git commit -m "message"</code>
Lire l'historique	Onglet History	<code>git log --oneline</code>

Je veux...	Dans GitHub Desktop	Commande Git
Envoyer mes commits	Push origin	<code>git push</code>
Regarder les nouveautés	Fetch origin	<code>git fetch</code>
Récupérer les nouveautés	Pull origin	<code>git pull</code>
Annuler des modifications non committées	Clic droit > Discard changes...	<code>git restore fichier</code>
Annuler un commit	History > clic droit > Revert changes in commit	<code>git revert identifiant</code>
Créer une branche	Current Branch > New Branch	<code>git switch -c nom</code>
Changer de branche	Current Branch > choisir	<code>git switch nom</code>
Fusionner une branche	Branch > Merge into current branch...	<code>git merge nom</code>

7. Consigne du projet n°1

Tous les groupes réalisent le même projet n°1, selon les mêmes règles de programmation et la même notation.

7.1 Sujet du projet n°1 : simulation du comportement d'un DAB (distributeur automatique de billets)

Écrivez un programme qui simule un distributeur de billets : l'utilisateur s'identifie avec un code PIN, consulte son solde et retire de l'argent. Le distributeur ne gère qu'un seul compte, défini au début du programme avec des valeurs de votre choix, par exemple un code PIN à 4 chiffres, un solde de 500 € et un plafond de 300 € par retrait. Il distribue des billets de 10, 20 et 50 €, en quantité illimitée.

Noyau minimal exigé (tous les groupes)

1. **Identification** : le programme demande le code PIN. Après 3 codes faux, la carte est bloquée et le programme s'arrête avec un message explicite.
2. **Menu** : une fois identifié, l'utilisateur choisit entre consulter le solde, retirer de l'argent ou quitter. Le menu réapparaît après chaque opération, jusqu'à ce que l'utilisateur quitte.
3. **Retrait** : le montant demandé est refusé, avec un message clair qui explique pourquoi, s'il n'est pas un entier positif, s'il n'est pas un multiple de 10 €, s'il dépasse le solde ou s'il dépasse le plafond de retrait. Sinon, le solde est mis à jour.

4. **Billets** : le distributeur calcule et affiche la composition du retrait en billets de 10, 20 et 50 €, avec le plus petit nombre de billets possible (exemples ci-dessous).
5. **Autres cas** : une saisie vide ou non numérique ne fait jamais planter le programme.
6. **Des tests** : une fonction `lancer_les_tests()` contenant des `assert` pour les fonctions de calcul (vérification du code PIN, vérification du montant, décomposition en billets), avec au moins les exemples ci-dessous et vos propres cas limites (par exemple 10, 60, 80, 290).
7. **Une documentation** : une docstring pour chaque fonction, et le fichier `README.md` du dépôt avec le but du programme, son mode d'emploi, des exemples et la répartition du travail dans le groupe.

Montant retiré	Billets distribués
10 €	1 billet de 10 €
30 €	1 billet de 20 €, 1 billet de 10 €
70 €	1 billet de 50 €, 1 billet de 20 €
90 €	1 billet de 50 €, 2 billets de 20 €
130 €	2 billets de 50 €, 1 billet de 20 €, 1 billet de 10 €
250 €	5 billets de 50 €

Les élèves proposeront d'eux-mêmes les pistes d'approfondissement, et les groupes pourront ainsi se différencier.

Date des exposés : du 22 au 26 novembre 2026.

7.2 Attendus concernant la programmation

Votre programme doit pouvoir être lu et compris par un être humain, même non averti. C'est l'exigence principale de ce projet, avec deux règles.

Règle 1 : des noms intelligibles. Les noms de fonctions et de variables disent ce qu'ils font ou ce qu'ils contiennent.

À éviter	À préférer
<code>def calc(1):</code>	<code>def calculer_moyenne(liste_de_notes):</code>
<code>x, tmp, r2</code>	<code>nombre_saisi, reste_de_la_division</code>
<code>def f(a, b, c):</code>	<code>def convertir_de_base_a_vers_base_b(nombre, base_a, base_b):</code>

Règle 2 : des fonctions qui appellent des fonctions. N'hésitez pas à introduire des niveaux dans les appels de fonctions, afin de rendre le programme le plus compréhensible possible. Deux versions du même programme vous sont fournies en exemple :

`bin_dec_hex__to__bin_dec_hex.py` (noms en anglais) et `bin_dec_hex__vers__bin_dec_hex.py` (noms en français). Elles montrent cette manière de procéder. Dans les deux, la fonction principale se lit comme une phrase. Version anglaise :

```
def bin_dec_hex__to__bin_dec_hex(number, cur_base, target_base):
    if not check_inputs(number, cur_base, target_base):
        return None
    clean_number = normalize_number(number, cur_base)
    value = any_to_decimal(clean_number, cur_base)
    result = decimal_to_any(value, target_base)
    return add_prefix(result, target_base)
```

Version française :

```
def bin_dec_hex__vers__bin_dec_hex(nombre, base_depart, base_arrivee):
    if not verifier_donnees(nombre, base_depart, base_arrivee):
        return None
    nombre_propre = normaliser_nombre(nombre, base_depart)
    valeur = convertir_vers_decimal(nombre_propre, base_depart)
    resultat = convertir_depuis_decimal(valeur, base_arrivee)
    return ajouter_prefixe(resultat, base_arrivee)
```

Chacune des fonctions appelées (`check_inputs`, `any_to_decimal` ...) appelle à son tour d'autres fonctions, qui en appellent d'autres encore, jusqu'à des fonctions de quelques lignes.

Dans ces deux fichiers, chaque nouvelle fonction introduite est écrite **tout en haut** : la fonction principale se trouve donc tout en bas. Le commentaire `# n°...` au-dessus de chaque fonction donne son numéro d'ordre d'écriture et la fonction qui l'a introduite. Lisez les fichiers en partant du bas, puis en remontant, pour retrouver la démarche suivie.

Méthode conseillée

1. Écrivez d'abord la fonction principale, en appelant des fonctions qui n'existent pas encore.
2. Pour chacune de ces fonctions, écrivez son rôle (docstring), puis son corps, qui appelle à son tour de nouvelles fonctions.
3. Continuez jusqu'à ce que chaque fonction soit assez simple pour être comprise d'un coup d'œil.
4. Testez chaque fonction séparément (avec des `assert`, par exemple), puis faites un commit.

Tester chaque fonction (point 4 de la méthode)

Tester une fonction, c'est vérifier qu'elle renvoie le bon résultat sur des exemples dont vous connaissez déjà la réponse. On écrit un test avec `assert` : si la condition est vraie, il ne se passe rien ; si elle est fausse, Python s'arrête et signale l'erreur (`AssertionError`), avec le numéro de la ligne en cause.

```
def est_bissextile(annee):
    return (annee % 4 == 0 and annee % 100 != 0) or annee % 400 == 0

def lancer_les_tests():
    assert est_bissextile(2024) == True
    assert est_bissextile(2023) == False
    assert est_bissextile(1900) == False    # siècle non divisible par 400
    assert est_bissextile(2000) == True     # siècle divisible par 400
    print("Tous les tests sont passés.")

lancer_les_tests()
```

1. **Testez chaque fonction juste après l'avoir écrite**, avant de passer à la suivante : une erreur se trouve bien plus facilement dans 5 lignes que dans 200.
2. **Testez les fonctions du bas de l'arbre d'appels en premier**, celles qui n'en appellent pas d'autres, puis celles qui les utilisent. Si une fonction du haut échoue alors que celles du bas réussissent, l'erreur est dans la fonction du haut.
3. **Prévoyez plusieurs exemples par fonction** : un cas ordinaire, un cas limite (0, une chaîne vide, la plus petite ou la plus grande valeur possible) et, si la fonction doit les gérer, un cas invalide.
4. **Calculez la réponse attendue à la main** avant d'écrire le test. Recopier le résultat affiché par votre fonction ne prouve rien.
5. **Regroupez vos tests dans une fonction `lancer_les_tests()`** et relancez-la après chaque modification : vous vérifiez ainsi que vous n'avez rien cassé.

6. **Quand un test échoue, corrigez la fonction**, pas le test, sauf si c'est la réponse attendue que vous aviez mal calculée. Vous pouvez ajouter un message : `assert est_bissextile(1900) == False, "1900 n'est pas bissextile"`.
7. **Faites un commit par fonction testée**, avec un message du type `Ajout de est_bissextile et de ses tests`.

7.3 Modalités de notation

La note du projet repose pour moitié sur l'exposé de votre groupe, et pour l'autre moitié sur l'évaluation de la production de chacun pendant les séances.

Composante	Part de la note	Comment
Exposé du groupe (8 min + 2 min de questions éventuelles)	50 %	Grille de critères (ci-dessous)
Production de chacun, évaluée après chaque séance de projet	2 × 25 %	Évaluation directe sur la plateforme ; la qualité passe avant la quantité

Exposé : les critères d'évaluation

L'évaluation s'appuie sur les six critères qui suivent. La note est sur 20.

Critère	Ce qui est évalué	Notes possibles
1. Documentation	La documentation du programme	0, 1, 2 ou 3
2. Clarté du code	Noms, structure, appels de fonctions	0, 1, 2 ou 3
3. Exécution du code	Le programme fonctionne	0, 1, 2 ou 4
4. Prise en compte des autres cas	Cas particuliers et données invalides	0, 1, 2 ou 3
5. Ergonomie de l'interface	Facilité d'utilisation du programme	0, 1, 2 ou 3
6. Clarté de l'exposé	Qualité de la présentation orale	0, 1, 2 ou 4

La colonne « Ce qui est évalué » résume l'intitulé de chaque critère : le fichier Excel fait foi.

Production de chacun : ce qui est évalué

Après chaque séance, la production de chaque membre du groupe est évaluée. Ce sont vos commits, signés de votre propre compte GitHub, qui en témoignent : commitez donc vos vraies contributions, avec des messages précis (voir l'étape 7 du parcours Git).

Interactions avec une IA : à documenter dans les commits

Toutes les interactions avec une IA doivent être documentées dans le commit correspondant, de la façon suivante :

1. Dans le message du commit (champ **Summary** de GitHub Desktop), écrivez : **IA: écriture de la fonction ma_fct.**
2. Dans le champ **Description** du commit, copiez-collez le rendu de l'IA et/ou sa description.
3. Toujours dans la **Description**, commentez ce rendu : niveau de pertinence, ce qui était juste, ce que vous avez corrigé ou rejeté.

Exemple de commit :

Summary :

IA: écriture de la fonction est_bissextile

Description :

Rendu de l'IA :

(copier-coller ici le code et/ou l'explication fournis par l'IA)

Commentaire :

Rendu pertinent à 80 % : la logique des années divisibles par 4 est juste, mais le cas des siècles était faux. J'ai corrigé la condition moi-même.